Booleans and Conditionals

Booleans

The `bool` Type

Python has a built-in variable type called bool, used to represent logical states.

It can only hold one of exactly two possible values:

`True` `False`

example.py

```
x = True
print(x)
print(type(x))
```

Boolean Operators

Rather than putting `True` or `False` directly in our code, we usually get boolean values from boolean operators. These are operators that answer yes/no questions.

Operation	Description
`a == b`	a equal to b
`a < b`	a less than b
`a <= b`	a less than or equal to b

Operation	Description
`a != b`	a not equal to b
`a > b`	a greater than b
`a >= b`	a greater than or equal to b

Returning Booleans

Conditional Logic

Functions can evaluate complex conditions and directly return a `bool` value.

19 Years Old:`False`

45 Years Old:`True`

president.py

```python
def can_run_for_president(age):
"""Can someone run for president?"""
return age >= 35

print(can_run_for_president(19))
# Output: False
print(can_run_for_president(45))
# Output: True
```

Try Out & Practice

Comparison

```
# Float vs Int
3.0 == 3
```

> True

```
# String vs Int
'3' == 3
```

> False

Boolean Casts

```
print(bool(1))
True

print(bool(0))
False

print(bool("Hi"))
True

print(bool(" "))
False
```

Logical Precedence

Evaluate the expression based on operator precedence rules (and before or):

```
True or True and False
```

Step 1: Evaluate 'and'

```
True and False False
```

Step 2: Evaluate 'or'

```
True or False True
```

Returning Booleans

Modulo & Booleans

The modulo operator `%` returns the remainder of division. Comparing it directly evaluates to a `bool`.

is_odd(100): `False`

is_odd(-1): `True`

is_odd.py

```python
def is_odd(n):
return (n % 2) == 1

print(is_odd(100))
# Output: False

print(is_odd(-1))
# Output: True
```

Conditionals

How Conditionals Work

Conditional statements, often referred to as *if-then* statements, let you control what pieces of code are run based on the value of some Boolean condition.

Boolean Core

Every decision branch starts with an expression that evaluates to either `True` or `False`.

The Keywords

Booleans are most useful when combined with statements using these primary keywords:

`if` Initial Condition

Evaluates the first expression to decide if the block should execute.

`elif` Alternative Checks

Short for "else if". Tests multiple conditions in sequence.

`else` Fallback Execution

Runs default code if all previous conditions evaluate to False.

Multi-Branch Conditionals

Branch Analysis

The function evaluates multiple conditions sequentially using `if-elif-else`. Once a condition evaluates to `True`, its block executes and the rest are skipped.

Tracing Output:

```
>>> inspect(0)
0 is zero

>>> inspect(-15)
-15 is negative
```

inspect.py

```python
def inspect(x):
  if x == 0:
    print(x, "is zero")
  elif x > 0:
    print(x, "is positive")
  elif x < 0:
    print(x, "is negative")
  else:
    print(x, "is unlike...")

inspect(0)
inspect(-15)
```

Truth Value Testing

0 is Falsy

In Python, 0 behaves like False. The first block is skipped, so it prints "spam".

falsy_check.py

```
if 0:
  print(0)
else "spam":
  print("spam")
```

Answer: "spam"

Non-Zero is Truthy

Any number other than 0 behaves like True. The code runs the first block and prints "Digits".

truthy_check.py

```
if 10:
  print("Digits")
else "spam":
  print("spam")
```

Answer: "Digits"